

Problem A

Acorn Quarrels

Time Limit: 1 second

There are N squirrels storing acorns in a tree. The (botanical) oak tree is also a graph-theoretic tree: a connected graph with N vertices labeled from 1 to N and $N - 1$ undirected edges. Each squirrel sits at a different vertex of the tree and two squirrels are *neighbors* if their vertices share an edge.

In ascending order of vertex label starting with the squirrel at vertex 1, each squirrel steals one acorn from the neighboring squirrel that currently has the most acorns. If multiple neighbors are tied for having the most acorns, the squirrel steals one acorn from each of them!



Paul Danese, CC BY-SA 4.0, via Wikimedia Commons

To limit the fallout of these shenanigans, you want to distribute acorns to the squirrels so that each squirrel begins with at least N acorns (so that no squirrel runs out of acorns due to thefts) and ends with the same number of acorns after all N squirrels are done stealing from each other as they had originally. It can be shown that such a distribution exists where every squirrel begins with at most $2N - 1$ acorns. Find any distribution satisfying these conditions.

Input

The first line of input contains an integer N ($2 \leq N \leq 10^5$), the number of vertices (and squirrels).

Each of the following $N - 1$ lines contains two space-separated integers u and v ($1 \leq u, v \leq N, u \neq v$), indicating that an edge exists between vertices u and v . There is at most one edge between any pair of vertices, and the edges form a tree.

Output

Print N space-separated integers $d_1, d_2, \dots, d_N$ satisfying $N \leq d_i \leq 2N - 1$, where d_i is the number of acorns you would like to distribute to the squirrel at vertex i . Your solution will be accepted if every squirrel ends with the same number of acorns as they started with after all N thefts have occurred. It can be proved that such a distribution of acorns always exists.



ICPC International Collegiate Programming Contest
**The 2026 ICPC North America
Championship**

HOSTED
BY



UNIVERSITY OF
CENTRAL FLORIDA

Sample Input 1

```
5
1 2
1 3
1 4
2 5
```

Sample Output 1

```
6 5 7 7 9
```

Sample Input 2

```
8
5 4
3 7
8 4
4 7
5 2
1 3
6 4
```

Sample Output 2

```
14 15 10 9 13 14 14 14
```

Problem B. Chain Prefix Rank (Hard Version)

Time Limit 5000 ms

Mem Limit 1048576 kB

This is the hard version of the problem. The difference between the versions is that in this version, $n \leq 5 \cdot 10^5$. You can hack only if you solved all versions of this problem.

Given a tree T with n vertices rooted at 1 and a sequence a of length n , count the number of permutations* p of length n that satisfy the following condition:

- For all $1 \leq u \leq n$, there are exactly a_u vertices v such that v is an ancestor of u in T and $p_v < p_u$.

Output the answer modulo 998 244 353. Input data is selected in such way, such that it's guaranteed that at least one valid permutation exists.

*A permutation of length n is an array consisting of n distinct integers from 1 to n in arbitrary order. For example, $[2, 3, 1, 5, 4]$ is a permutation, but $[1, 2, 2]$ is not a permutation (2 appears twice in the array), and $[1, 3, 4]$ is also not a permutation ($n = 3$ but there is 4 in the array).

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 5000$). The description of the test cases follows.

The first line of each test case contains an integer n ($2 \leq n \leq 5 \cdot 10^5$) — the number of vertices.

The second line of each test case contains $n - 1$ integers $fa_2, fa_3, \dots, fa_n$ ($1 \leq fa_i < i$) — fa_i is the parent of i on T .

The third line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < n$).

It is guaranteed that the sum of n over all test cases does not exceed $5 \cdot 10^5$. Input data is selected in such way, such that it's guaranteed that at least one valid permutation exists.

Output

For each test case, output one integer — the number of permutations modulo 998 244 353.

Examples

Input	Output
3 5 1 2 3 4 0 1 2 3 4 5 1 2 1 1 0 1 0 0 0 8 1 1 3 3 4 5 7 0 0 1 0 1 3 3 1	1 6 4

Note

[Visualizer link](#)

In the first test case, the only permutation which satisfies the condition is $[1, 2, 3, 4, 5]$.

In the second test case, permutations which satisfy the condition are

$[4, 5, 1, 2, 3]$, $[4, 5, 1, 3, 2]$, $[4, 5, 2, 1, 3]$, $[4, 5, 2, 3, 1]$, $[4, 5, 3, 1, 2]$, $[4, 5, 3, 2, 1]$.

In the third test case, permutations which satisfy the condition are

$[3, 1, 6, 2, 5, 7, 8, 4]$, $[3, 1, 6, 2, 5, 8, 7, 4]$, $[3, 2, 6, 1, 5, 7, 8, 4]$, $[3, 2, 6, 1, 5, 8, 7, 4]$.

Problem C. Bowser's Castle (Hard Version)

Time Limit 8000 ms

Mem Limit 262144 kB

[Final Boss Phase 2 \(Lava\) / Giant Bowser — Shiho Fujii, Ryo Nagamatsu, New Super Mario Bros. Wii](#)



This is the hard version of the problem. The difference between the versions is that in this version, $n \leq 400$ and you may make 1500 queries. You can hack only if you solved all versions of this problem.

This is an interactive problem.

Let n be a positive integer. A function on n variables $x_1, \dots, x_n$ is called *min-max* if it can be constructed as writing only min and max calls around $x_1, \dots, x_n$ with each variable appearing exactly once in order from left to right in the expression. For example, $\min(x_1, x_2, x_3)$ is a min-max function on 3 variables, but $\max(\min(x_1, x_3), x_2)$ and $\min(\max(x_1, x_2), \max(x_2, x_3))$ are not.

Bowser has chosen a min-max function on n ($2 \leq n \leq 400$) variables and tells you n . Additionally, he lets you ask queries of the following form: you give Bowser n integers $x_1, \dots, x_n$ ($1 \leq x_i \leq 10^9$) as input, and he tells you $f(x_1, \dots, x_n)$.

To escape his castle, you have to deduce his function using at most 1500 queries in total. After that, to prove to him that you have learned the function, Bowser will give you up to

5000 of his own inputs $x_1, \dots, x_n$, to which you must respond with the correct value of $f(x_1, \dots, x_n)$.

Since Bowser's castle is very secure, he actually has multiple functions for you to figure out, with a total variable count of at most 400. Additionally, he constrains that you use at most 1500 queries across all of the functions and that he will also not ask more than 5000 queries in total.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 200$). The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \leq n \leq 400$) — the number of variables in the min-max function.

It is guaranteed that the sum of n over all test cases does not exceed 400.

After you read this line of input, the interaction begins with your first query.

Interaction

To make a query, output a line (without quotes) in the following format:

- `"? x1 x2 ... xn"` ($1 \leq x_i \leq 10^9$).

Then, after each query, read a single integer — the answer to your query. You can make up to 1500 queries of this type **in total across all t test cases**.

Once you are ready to begin answering Bowser's queries, output a line containing a single character `!"` (without quotes).

Each of Bowser's queries contains a single line of n integers $x_1, x_2, \dots, x_n$ ($1 \leq x_i \leq 10^9$) — Bowser's input. It is guaranteed that Bowser will not make more than 5000 queries **in total across all t test cases**.

After each query, output a single integer — the value of $f(x_1, x_2, \dots, x_n)$. Note that these queries are **online**, so Bowser will only provide you the next query after you respond to his previous query.

Once Bowser is done asking queries, you will receive a single line containing the integer 0. After this, proceed to the next test case or exit if this is the last test case.

The interactor in this task is **partially adaptive**. In particular, the functions Bowser chooses are fixed at the beginning of each interaction, but his queries to you may change.

After printing each query do not forget to output the end of line and flush* the output. Otherwise, you will get `Idleness limit exceeded` verdict. If, at any interaction step, you read `-1` instead of valid data, your solution must exit immediately. This means that your solution will receive `Wrong answer` because of an invalid query or any other mistake. Failing to exit can result in an arbitrary verdict because your solution will continue to read from a closed stream.

Hacks

To hack, use the following format.

The first line should contain a single integer t ($1 \leq t \leq 200$) — the number of test cases.

The first line of each test case should contain a single integer n ($2 \leq n \leq 400$) — the number of variables in the min-max function.

The second line of each test case should contain a series of space-separated tokens (without quotes) from the following list, specifying the min-max function:

- `"min"`
- `"max"`
- `"("`
- `)"`
- `","`
- positive integers in the range $1, 2, \dots, n$.

The sum of n over all test cases should not exceed 400.

For instance, the hack case corresponding to the example input is as follows:

```

1 | 2
2 | 3
3 | max ( 1 , 2 , 3 )
4 | 4
5 | min ( max ( 1 , 2 ) , max ( 3 , 4 ) )
```

*To flush, use:

- `fflush(stdout)` or `cout.flush()` in C++;
- `sys.stdout.flush()` in Python;

- see the documentation for other languages.

Examples

Input	Output
2 3	? 5 4 3
5	? 1 2 1
2	!
3 6 2	6
3 5 7	7
0 4	? 1 9 8 7
8	? 5 1000000000 2 3
3	!
8 7 6 5	6
0	

Note

In the first test case, Bowser's min-max function is $f := \max(x_1, x_2, x_3)$. After asking two queries to receive $f(5, 4, 3) = 5$ and $f(1, 2, 1) = 2$ from the jury, the solution is ready to answer the jury's queries. It correctly deduces that $f(3, 6, 2) = 6$ and $f(3, 5, 7) = 7$.

In the second test case, Bowser's min-max function is $f := \min(\max(x_1, x_2), \max(x_3, x_4))$. After asking two queries to receive $f(1, 9, 8, 7) = 8$ and $f(5, 10^9, 2, 3) = 3$ from the jury, the solution is ready to answer the jury's queries. It correctly deduces that $f(8, 7, 6, 5) = 6$.

Note that it is not guaranteed that the queries in the example uniquely determine the min-max function. The example is there only to demonstrate how the queries work. Empty lines in the example input and output are given only for better readability; you don't need to output them in your solution.

2017 Canadian Computing Olympiad

Day 2, Problem 1

Rainfall Capture

Time Limit: 1 second

Problem Description

It was a dark and stormy night. It also rained, and rained, and rained.

Lucy wants to capture some of the rain, but she only has limited materials. She has a collection of pillars, of various heights, which she can configure to capture the rain. Each pillar is an integer height and has length and width of 1. Once Lucy has her configuration of pillars, she has enough other siding material to enclose the front and back to allow rain to fill the all the available space in between pillars. There is more than enough rain and any excess rain will overflow and get absorbed into the earth.

For example, if Lucy has pillars of height 1, 5, 2, 1, 4, she could configure them as follows (all configurations are illustrated from the side):

```
  *
 *  *
 *  *
** *
*****
```

which would capture 5 units of rain (R) as follows:

```
  *
 *RR*
 *RR*
 **R*
*****
```

For this first collection of pillars (1, 5, 2, 1, 4), she could also capture 6 units of rain as follows:

```
  *
 *RR*
 *RR*
**RR*
*****
```

As another example, if the collection of pillars was 5, 1, 5, 1, 5, Lucy could capture 8 units of rain as follows:

```

    *R*R*
    *R*R*
    *R*R*
    *R*R*
    *****

```

Finally, this configuration of (5, 1, 4, 1, 5) captures 9 units of rain:

```

    *RRR*
    *R*R*
    *R*R*
    *R*R*
    *****

```

Lucy has N pillars ($2 \leq N \leq 500$) with heights $h_1, h_2 \dots h_N$ ($1 \leq h_i \leq 50$). She would like to know, of all possible configurations of pillars, what are all of the obtainable volumes of rainfall that she can capture using these N pillars.

Input Specification

The first line contains the integer N ($2 \leq N \leq 500$) signifying the number of pillars. The next line contains the integers h_i ($1 \leq h_i \leq 50, 1 \leq i \leq N$), representing the i th pillar height.

For 5 of the 25 marks available, $N \leq 10$.

For an additional 10 of the 25 marks available, $N \leq 50$.

Output Specification

On one line, output a space-separated list of all possible obtainable integer volumes of rain captured, in increasing order.

Sample Input 1

```

5
1 5 2 1 4

```

Output for Sample Input 1

```

0 1 2 3 4 5 6 8

```

Explanation for Output for Sample Input 1

This is the first given example.

Sample Input 2

5
5 1 5 1 5

Output for Sample Input 2

0 4 8

Explanation for Output for Sample Input 2

This is the second given example.

Sample Input 3

5
5 1 4 1 5

Output for Sample Input 3

0 1 3 4 5 6 7 8 9

Explanation for Output for Sample Input 3

This is the third given example.



Security Guard

In JOI Kingdom, there are N islands, numbered from 1 to N . Each island has the **insecurity level**. The insecurity level of the island i ($1 \leq i \leq N$) is S_i .

In JOI Kingdom, ships between pairs of islands are mostly used as the methods of transportations. There are M ships, numbered from 1 to M . The ship j ($1 \leq j \leq M$) connects the island A_j and the island B_j . We can run ships when necessary. It is possible to travel from any island to any other island by taking a number of ships.

In JOI Kingdom, there is a plan to introduce new ships. We can choose any pairs of islands where newly introduced ships connect.

One day, an incident occurred. A ship at anchor was attacked. Prime minister K of JOI Kingdom decided to introduce new ships. He also demands that ships in JOI Kingdom should satisfy the following **Security Condition**.

- When a ship is anchored at the island i ($1 \leq i \leq N$), the number of security guards on the ship is greater than or equal to S_i .

However, since it is expensive to hire security guards, we want to minimize the number of hired security guards. As long as the condition “it is possible to travel from any island to any other island by taking a number of ships” is satisfied, it is possible to abolish ships which are currently running.

Therefore, we will run ships as follows. Here, k is the number of newly introduced ships.

1. For each of the k newly introduced ships, we choose two islands where it connects.
2. We choose a number of (more than or equal to 0) ships, and we abolish them. It is allowed to abolish newly introduced ships.
3. For each of the ships, we anchor it at one of the two islands where it connects. We make a number of security guards get on it. Moreover, the following conditions should be satisfied.

Condition For every pair u, v ($1 \leq u \leq N$, $1 \leq v \leq N$) of islands, it is possible to transport a passenger from the island u to the island v by repeating the following operations a number of times. In the process, Security Condition should be satisfied all the time.

- We make a passenger or security guards get on a ship which is anchored at the island where the passenger or security guards are staying.
- We make a passenger or security guards get off a ship at the island where the ship is currently anchored.
- We move a ship from the island where the ship is currently anchored to the other island where the ship connects.



Since the budget is limited, we can introduce at most Q new ships. For each k ($0 \leq k \leq Q$), Prime minister K wants to know the minimum possible number of hired security guards if the number of newly introduced ships is k .

Write a program which, given the information of islands and the routes of the ships and the number of new ships we can introduce, calculates the minimum possible number of hired security guards for each k .

Input

Read the following data from the standard input.

N M Q
 S_1 S_2 $\cdots$ S_N
 A_1 B_1
 A_2 B_2
 $\vdots$
 A_M B_M

Output

Write $Q+1$ lines to the standard output. The $(k+1)$ -th line ($0 \leq k \leq Q$) of output should contain the minimum possible number of hired security guards if the number of newly introduced ships is k .

Constraints

- $2 \leq N \leq 200\,000$.
- $N-1 \leq M \leq 400\,000$.
- $0 \leq Q \leq 200\,000$.
- $1 \leq S_i \leq 10^9$ ($1 \leq i \leq N$).
- $1 \leq A_j < B_j \leq N$ ($1 \leq j \leq M$).
- $(A_x, B_x) \neq (A_y, B_y)$ ($1 \leq x < y \leq M$).
- It is possible to travel from any island to any other island by taking a number of ships.
- Given values are all integers.



Subtasks

1. (12 points) $M = N - 1$, $Q = 0$, $S_i \leq 2$ ($1 \leq i \leq N$), $A_j = j$, $B_j = j + 1$ ($1 \leq j \leq M$).
2. (13 points) $M = N - 1$, $Q = 0$, $A_j = j$, $B_j = j + 1$ ($1 \leq j \leq M$).
3. (12 points) $M = N - 1$, $Q = 0$.
4. (13 points) $Q = 0$.
5. (8 points) $N \leq 16$.
6. (18 points) $N \leq 3\,000$.
7. (24 points) No additional constraints.

Sample Input and Output

Sample Input 1	Sample Output 1
4 3 0 2 1 3 2 1 2 2 3 3 4	7

If the number of newly introduced ships is 0, we need 7 security guards. For example, the conditions are satisfied if we allocate the ships and 7 security guards as follows.

- The ship 1 is initially anchored at the island 2, and two security guards get on the ship 1.
- The ship 2 is initially anchored at the island 2, and two security guards get on the ship 2.
- The ship 3 is initially anchored at the island 4, and three security guards get on the ship 3.

Let us explain how to transport a passenger in the following two cases.

- We transport a passenger from the island 1 to the island 4.
- We transport a passenger from the island 3 to the island 2.

We can transport a passenger from the island 1 to the island 4 as follows. The islands where the ships 1, 2, 3 are anchored, and the numbers of security guards on the ships 1, 2, 3 are written in this order. The numbers of security guards on the islands 1, 2, 3, 4 are written in this order.



#	Operation	The islands where the ships are anchored	The numbers of security guards on the ships	The numbers of security guards on the islands
-	-	2, 2, 4	2, 2, 3	0, 0, 0, 0
1	Move the ship 1 from the island 2 to the island 1	1, 2, 4	2, 2, 3	0, 0, 0, 0
2	Make a passenger get on the ship 1	1, 2, 4	2, 2, 3	0, 0, 0, 0
3	Move the ship 1 from the island 1 to the island 2	2, 2, 4	2, 2, 3	0, 0, 0, 0
4	Make one security guard and the passenger get off the ship 1	2, 2, 4	1, 2, 3	0, 1, 0, 0
5	Make one security guard and the passenger get on the ship 2	2, 2, 4	1, 3, 3	0, 0, 0, 0
6	Move the ship 2 from the island 2 to the island 3	2, 3, 4	1, 3, 3	0, 0, 0, 0
7	Make the passenger get off the ship 2	2, 3, 4	1, 3, 3	0, 0, 0, 0
8	Move the ship 3 from the island 4 to the island 3	2, 3, 3	1, 3, 3	0, 0, 0, 0
9	Make the passenger get on the ship 3	2, 3, 3	1, 3, 3	0, 0, 0, 0
10	Move the ship 3 from the island 3 to the island 4	2, 3, 4	1, 3, 3	0, 0, 0, 0
11	Make the passenger get off the ship 3	2, 3, 4	1, 3, 3	0, 0, 0, 0

We can transport a passenger from the island 3 to the island 2 as follows.

#	Operation	The islands where the ships are anchored	The numbers of security guards on the ships	The numbers of security guards on the islands
-	-	2, 2, 4	2, 2, 3	0, 0, 0, 0
1	Make one security guard get off the ship 1	2, 2, 4	1, 2, 3	0, 1, 0, 0
2	Make one security guard get on the ship 2	2, 2, 4	1, 3, 3	0, 0, 0, 0
3	Move the ship 2 from the island 2 to the island 3	2, 3, 4	1, 3, 3	0, 0, 0, 0
4	Make a passenger get on the ship 2	2, 3, 4	1, 3, 3	0, 0, 0, 0
5	Move the ship 2 from the island 3 to the island 2	2, 2, 4	1, 3, 3	0, 0, 0, 0
6	Make the passenger get off the ship 2	2, 2, 4	1, 3, 3	0, 0, 0, 0



Since it is impossible to satisfy the conditions if the number of security guards is less than or equal to 6, output 7.

This sample input satisfies the constraints of Subtasks 2, 3, 4, 5, 6, 7.

Sample Input 2	Sample Output 2
4 3 1	7
2 1 3 2	5
1 2	
2 3	
3 4	

If the number of newly introduced ships is 0, similarly as Sample Input 1, we need 7 security guards.

If the number of newly introduced ships is 1, we need 5 security guards. For example, the conditions are satisfied if we allocate the ships and 5 security guards as follows.

- We introduce a new ship connecting the island 2 and the island 4. (In the following, we call it the ship 4.)
- We abolish the ship 3.
- We initially anchor the ship 1 at the island 2, and make two security guards get on the ship 1.
- We initially anchor the ship 2 at the island 2, and make one security guard get on the ship 2.
- We initially anchor the ship 4 at the island 2, and make two security guards get on the ship 4.

This sample input satisfies the constraints of Subtasks 5, 6, 7.

Sample Input 3	Sample Output 3
3 3 0	2
1 1 1	
1 2	
1 3	
2 3	

If the number of newly introduced ships is 0, we need 2 security guards. For example, the conditions are satisfied if we allocate the ships and 2 security guards as follows.

- We abolish the ship 3.
- We initially anchor the ship 1 at the island 1, and make one security guard get on the ship 1.
- We initially anchor the ship 2 at the island 1, and make one security guard get on the ship 2.



This sample input satisfies the constraints of Subtasks 4, 5, 6, 7.

Sample Input 4	Sample Output 4
8 7 0 2 2 2 2 2 2 2 2 1 2 2 3 3 4 4 5 5 6 6 7 7 8	14

This sample input satisfies the constraints of all the subtasks.

Sample Input 5	Sample Output 5
8 7 0 16 39 36 23 15 48 23 56 1 2 1 3 2 4 2 5 3 6 3 7 7 8	245

This sample input satisfies the constraints of Subtasks 3, 4, 5, 6, 7.



The 22nd Japanese Olympiad in Informatics (JOI 2022/2023)
Spring Training/Qualifying Trial
March 18–22, 2023 (Komaba, Tokyo)

Contest 4 – Security Guard

Sample Input 6	Sample Output 6
10 13 4	3139
314 159 265 358 979 323 846 264 338 327	2901
1 2	2722
1 4	2567
2 3	2461
2 5	
3 6	
4 5	
4 7	
5 6	
5 8	
6 9	
7 8	
8 9	
9 10	

This sample input satisfies the constraints of Subtasks 5, 6, 7.

Problem F. Wise Men (Hard Version)

Time Limit 4000 ms

Mem Limit 524288 kB

This is the hard version of the problem. The difference is constraints on the number of wise men and the time limit. You can make hacks only if all versions of this task are solved.

n wise men live in a beautiful city. Some of them know each other.

For each of the $n!$ possible permutations $p_1, p_2, \dots, p_n$ of the wise men, let's generate a binary string of length $n - 1$: for each $1 \leq i < n$ set $s_i = 1$ if p_i and p_{i+1} know each other, and $s_i = 0$ otherwise.

For all possible 2^{n-1} binary strings, find the number of permutations that produce this binary string.

Input

The first line of input contains one integer n ($2 \leq n \leq 18$) — the number of wise men in the city.

The next n lines contain a binary string of length n each, such that the j -th character of the i -th string is equal to '1' if wise man i knows wise man j , and equals '0' otherwise.

It is guaranteed that if the i -th man knows the j -th man, then the j -th man knows i -th man and no man knows himself.

Output

Print 2^{n-1} space-separated integers. For each $0 \leq x < 2^{n-1}$:

- Let's consider a string s of length $n - 1$, such that $s_i = \lfloor \frac{x}{2^{i-1}} \rfloor \bmod 2$ for all $1 \leq i \leq n - 1$.
- The $(x + 1)$ -th number should be equal to the required answer for s .

Examples

Input	Output
3 011 101 110	0 0 0 6

Input	Output
4 0101 1000 0001 1010	2 2 6 2 2 6 2 2

Note

In the first test, each wise man knows each other, so every permutation will produce the string 11.

In the second test:

- If $p = \{1, 2, 3, 4\}$, the produced string is 101, because wise men 1 and 2 know each other, 2 and 3 don't know each other, and 3 and 4 know each other;
- If $p = \{4, 1, 2, 3\}$, the produced string is 110, because wise men 1 and 4 know each other, 1 and 2 know each other and 2, and 3 don't know each other;
- If $p = \{1, 3, 2, 4\}$, the produced string is 000, because wise men 1 and 3 don't know each other, 3 and 2 don't know each other, and 2 and 4 don't know each other.